

Object-Oriented Programming

(3168-001)

Object–Oriented Programming (OOP)

Course Overview & Setup

Prof. Jaehyung Seo

(seojae777@konkuk.ac.kr)

Language Intelligence & Representation Lab.



▪ **Jaehyung Seo (서재형)**

- **Assistant Professor** @ *Computer Science and Engineering Dept.*

Natural Language Processing, Language modeling, Reasoning, Hallucination, Model Editing, Multimodal Understanding



Korea University (B.S., M.S., Ph.D.)

- **2026:** Published **2 papers** at a top-tier CV conference (CVPR)
- **2025:** Published **9 papers** at top-tier CS/AI/NLP conferences (ICLR, ACL, EMNLP ...)
- **2024:** Published **7 papers** at top-tier CS/AI/NLP conferences (ACL, EMNLP, EACL ...)
- **2023:** Published **5 papers** at top-tier CS/AI/NLP conferences (ACL, EMNLP, IJCNLP....)
- **2022:** Published **5 papers** at top-tier CS/AI/NLP conferences (NAACL, AACL, COLING ...)
- **2021-2025:** Published **+15 papers** in Q1–Q2 Computer Science journals
- Industry collaborations with Samsung Research, Samsung Fire & Marine Insurance, Samsung SDS, KT, Hyundai Motor Company, NCAI, Upstage, and NAVER
- International collaborations with A*STAR (Singapore), Microsoft Research Asia and York University (Canada)
- BEST PAPER Award (2024, 2025), NAVER Ph. D Fellowship Award (2022)
- Sovereign AI (VAETKI): National Representative AI Research Team (2025)

▪ **E-mail:** seojae777@konkuk.ac.kr

▪ Office: 공학관A 1409-1

▪ The fastest way to contact me is by email!

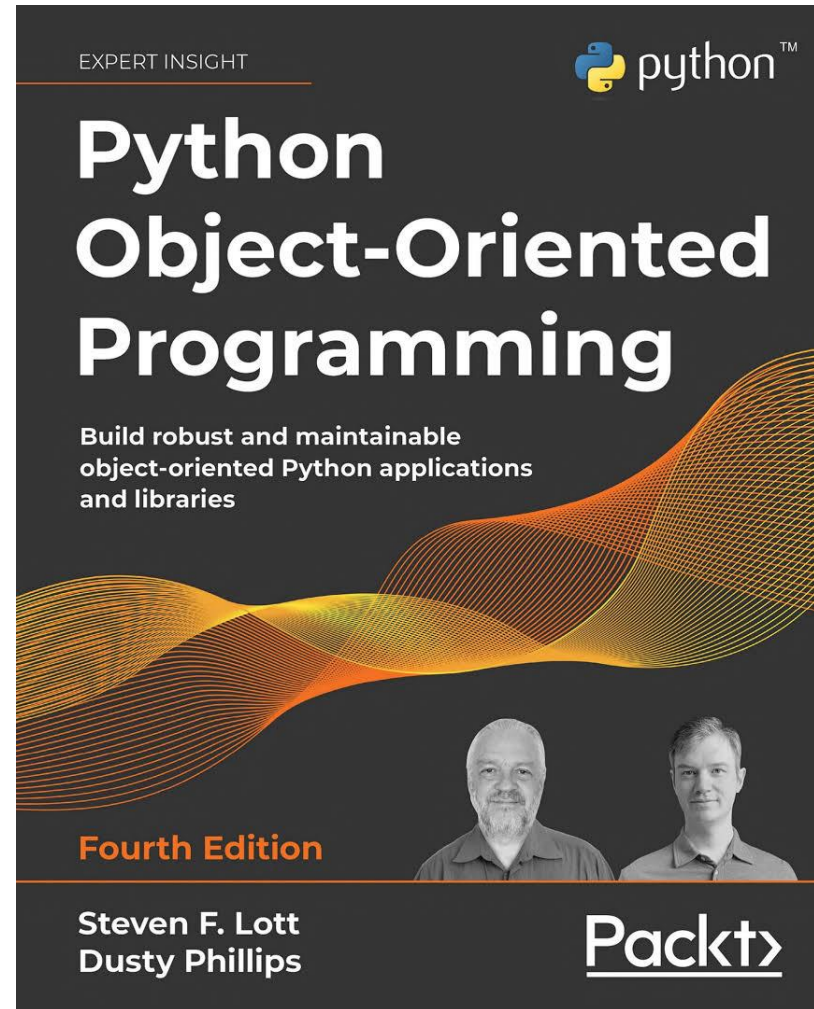
Schedule

- Mondays & Wednesdays (월/수)
 - (1) 9:00 – 9:50
 - (2) 10:00 – 10:50
 - Lecture Room: 새천년관 403호
 - Official Language: English (Korean QA)
-
- All the announcements and lecture slides will be posted via **eCampus**

Topics

- Introduction to OOP & Python; Setup (VS Code)
- Python Basic I
- Python Basic II
- OOP I
- OOP II
- OOP III
- OOP IV
- OOP for AI + Concurrency
- Numerical Computing with Numpy
- Data Management with Pandas
- Scikit-learn
- PyTorch I
- PyTorch II
- PyTorch III

Reference Book



Grading Policy

▪ Exam (70%)

- Midterm Exam (30%)
- Final Exam (40%)

- ✓ If a student is unable to attend an exam due to an **inevitable and officially acceptable reason**, the following score will be assigned: **Min (student's overall course average, class average of the exam)**.
- ✓ Additionally, extra report assignments may be required
- ✓ *Inevitable reasons*” **do NOT include** minor illness or personal inconvenience
- ✓ If the stated reason is found to be false, the student will receive an **“F”** for the course

▪ Assignment (20%)

- Assignment 1: Python Programming Basic (10%) Due date: **April 03**
- Assignment 2: Object-Oriented System Design (10%) Due date: **June 05**

▪ **Assignment 1: Python Programming Basic (10%) Due date: April 03**

- **Topic: Deep Dive into Python OOP Concepts**

- Select **ONLY ONE concept** (e.g., Class, Decorator, Data Structure, etc.) and analyze it in depth

- **Length Requirement:** Maximum 4 pages

- **Submit it any format!**

- **Required Components:**

(1) **Concept Explanation:** Theoretical background and necessity of the chosen OOP concept

(2) **Comparative Analysis:** The differences between applying and not applying the concept

(3) **Experimental:** Original example code designed by you to support your report

(4) **Results & Discussion:** Code execution results and your insights

- **Submission:** Please submit your assignment as a **PDF file** on **eCampus**

- **Recommendation Tool:**



▪ Assignment 2: Object-Oriented System Design (10%) Due date: June 05

Topic: Design and Implement an OOP System for Your Future Career/Interest

- You must practically **implement** an object-oriented system related to your future goals and write a comprehensive **report** detailing your project.

- **Length Requirement:** 4 to 8 pages (Minimum 4, Maximum 8)

- Required Components:

(1) **Introduction:** Motivation and objective of the chosen topic.

(2) **Related Work:** Existing systems or concepts related to your project.

(3) **Proposed OOP System:** System architecture, class design, and core implementation details.

(4) **Implementation & Usage / Results:** How the implemented system actually works (include execution screens or key code snippets).

(5) **Limitations:** Current shortcomings and potential future improvements.

(6) **Conclusion:** Summary of the project and lessons learned.

- **Submission:** Please submit your **report as a PDF file** on **eCampus**.

*(Optional) You may include a **GitHub link** in your report to share your source code.*

Grading Policy

- **Attendance** (10%)

University policy requires students to attend at least 2/3 of the scheduled classes. Otherwise, you'll fail this course.

- **Late Arrivals:** 2 Lates = 1 Absence

Attendance checks will be conducted at random times

- **Absences & Penalties:**

Up to 1 absences: No penalty (Full score)

From the 2nd absence: **-1 point** per absence

- **Excused Absences / Lateness:**

Accepted for valid or unavoidable reasons.

Please email valid proof to: **seojae777@konkuk.ac.kr**

- **Midterm (Wed, April 22) 09:00 – 11:00**

- Most questions will be based on the course materials and my lecture notes
- Question formats will primarily include True/False (+1 for correct, -1 for incorrect, 0 for blank), Multiple Choice, and Free-Response (or Short Answer)
- Answers may be written in English/Korean
- Scope: All materials covered before the midterm
- **In person written exam (closed book, no devices)**

- **Final Exam (Mon, June 22) 09:00 – 11:00**
 - Most questions will be based on the course materials and my lecture notes
 - Question formats will primarily include True/False (+1 for correct, -1 for incorrect, 0 for blank), Multiple Choice, and Free-Response (or Short Answer)
 - Answers may be written in English/Korean
 - Scope: All materials covered after the midterm
 - **In person written exam (closed book, no devices)**

- **Online Lecture (Pre-recorded)**

- **March 23**
- **March 25**
- **June 3**
- **June 8**
- Please watch the videos via **Konkuk eCampus**.
- Attendance is automatically checked when you complete the required viewing time within the designated learning period.

- **No Class**

- **May 25 (Buddha's Birthday)**

■ Setup

The screenshot shows the GitHub repository page for 'ElionLAB / OOP_2026_Practice'. The repository is public and has 0 stars, 0 forks, and 0 watchers. The main branch is 'main'. The repository contains the following files and folders:

File/Folder	Description	Commit Time
ch_01/src	Add Ch.01 exercise solutions notebook	2 hours ago
docs	Add macOS setup guides and cross-link all guide variants	1 hour ago
tests	Initial commit: 프로젝트 구조 및 환경 설정 가이드	2 hours ago
.gitignore	Update .gitignore for internal files	1 hour ago
README.md	Initial commit: 프로젝트 구조 및 환경 설정 가이드	2 hours ago

The README file is titled 'OOP 2026 Practice' and describes the repository as a 'Python Object-Oriented Programming 실습 저장소' (Python Object-Oriented Programming practice storage). It includes a link to the textbook 'Python Object-Oriented Programming, 4th Edition (Packt)' and a directory structure:

```
OOP_2026_Practice/
├── docs/          # 설치 가이드 등 문서
├── tests/         # 환경 검증 테스트
├── ch_01/
│   ├── src/
│   └── tests/
```

The right sidebar shows the repository's statistics and suggested workflows. The 'Languages' section shows that the repository is primarily composed of Jupyter Notebook files (86.7%) and Python files (13.3%).

https://github.com/ElionLAB/OOP_2026_Practice

Object–Oriented Programming (OOP)

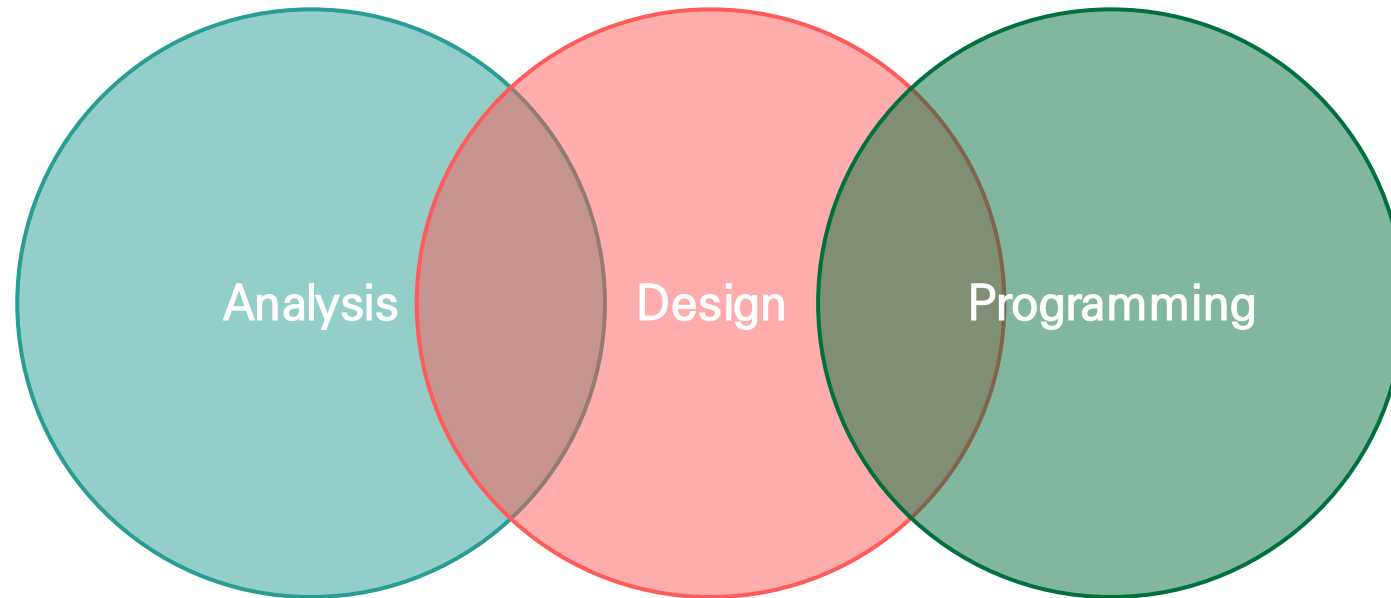
Lecture 1: Introduction to OOP

Prof. Jaehyung Seo

(seojae777@konkuk.ac.kr)

Language Intelligence & Representation Lab.

The Software Development Lifecycle



The traditional "waterfall" model separates these stages, but in reality, they overlap, combine, and interweave continuously.

What is an Object?

Definition

An object is a collection of **data** and associated **behaviors**.

Example

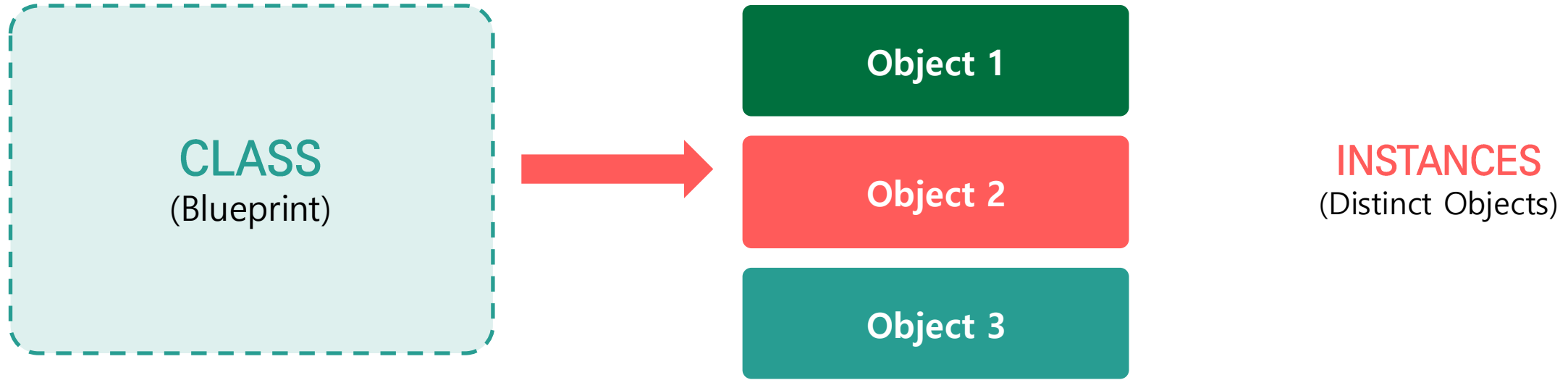
A baby's shape-sorting toy. The blocks (objects) have distinct **shapes** (data) and can be dropped into matching holes or trigger **bells** (behaviors).



OOA vs. OOD vs. OOP

Phase	Focus	Output
Object-Oriented Analysis (OOA)	What needs to be done.	System requirements / Identified objects.
Object-Oriented Design (OOD)	How it should be done.	Implementation specifications / Interfaces.
Object-Oriented Programming (OOP)	Doing the work.	A working, functional program.

Objects vs. Classes



A class is a blueprint that describes a kind of object.
An **object** is a specific, distinct **instance** of that class.



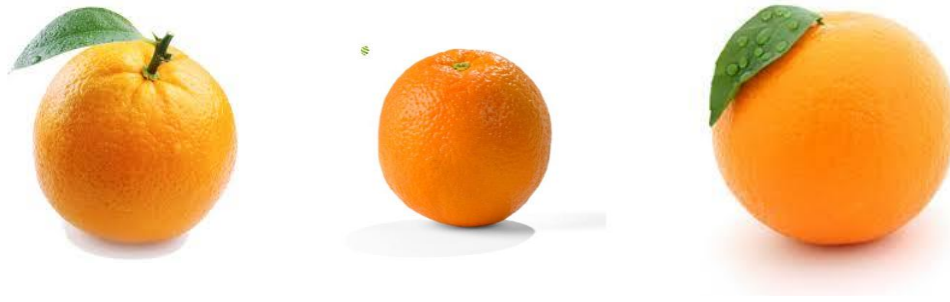
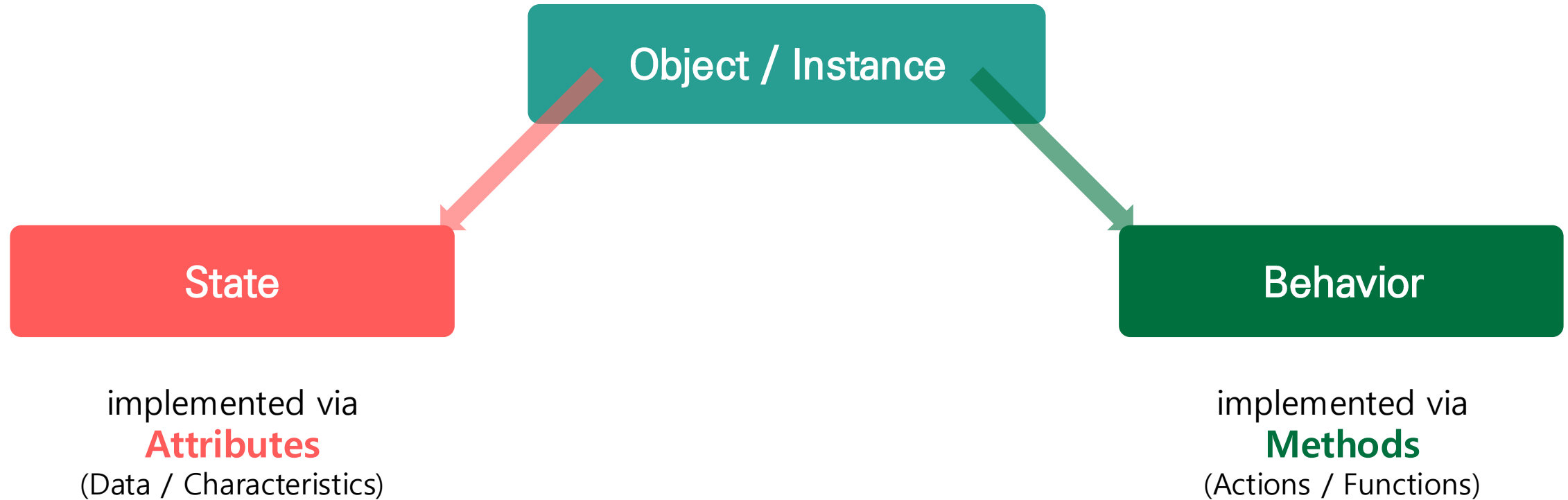
UML Class Associations



UML Multiplicity

- UML (Unified Modeling Language) allows us to diagram the multiplicity of relationships.
- Here, many (*) oranges go into one (1) basket.

Inside the Object



Object Attributes

Definition

Attributes represent the individual characteristics of a certain object. They define the object's current **state**.



Example

An **Apple** class might have attributes like **color** and **weight**. Every apple has a weight, but the specific value differs per instance.



Attributes in Action

Code Snippet:

A simple class showing state variables (attributes)

```
class Apple:
    def __init__(self, color: str, weight: float):
        # These are attributes describing the state
        self.color = color
        self.weight = weight
        self.barrel = None # Association attribute

apple1 = Apple("Red", 0.3)
apple2 = Apple("Green", 0.2)
```

Object Methods

Definition

Behaviors are **actions** that can occur on an object. They are programmed as **methods**.

Example

A Basket object might have a **sell()** behavior, or an Orange might have a **squeeze()** behavior that returns juice.



Methods in Action

Code Snippet:

Code showing methods altering object state

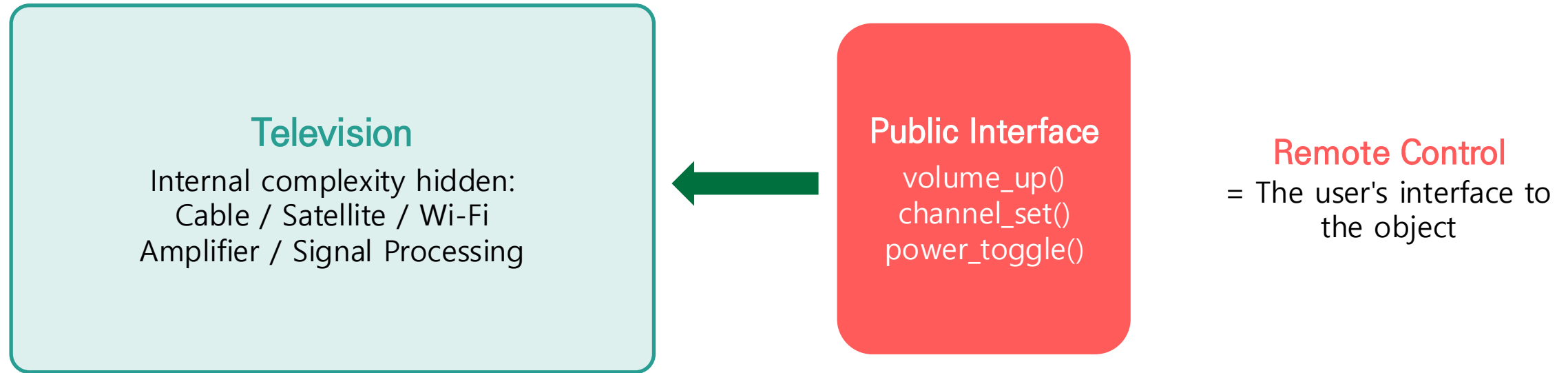
```
class Basket:
    def __init__(self, location: str):
        self.location = location
        self.oranges = []

    def add_orange(self, orange: Orange) -> None:
        # Behavior modifying internal state
        self.oranges.append(orange)

    def sell(self, customer: Customer) -> None:
        # Complex interaction with another object
        customer.bill(self.calculate_price())
        self.oranges.clear()

    ...
```

The Television Interface



The remote control is the **Public Interface**. The user presses a button (calls a method) without needing to know if the TV uses cable, satellite, or Wi-Fi internally.

Abstraction & Encapsulation

Key Takeaway

Information hiding (**encapsulation**) protects the internal state, while **abstraction** simplifies the model.

By keeping attributes **private** and exposing only carefully chosen **methods**, we ensure that external objects cannot accidentally corrupt the state of our object.

The public interface must be carefully designed to be **stable**, because any changes will break the client code.

Violating Privacy

The Danger

Accessing internal attributes directly from outside the class.

```
tv.internal_amplifier_voltage = 1000 (Boom!)
```

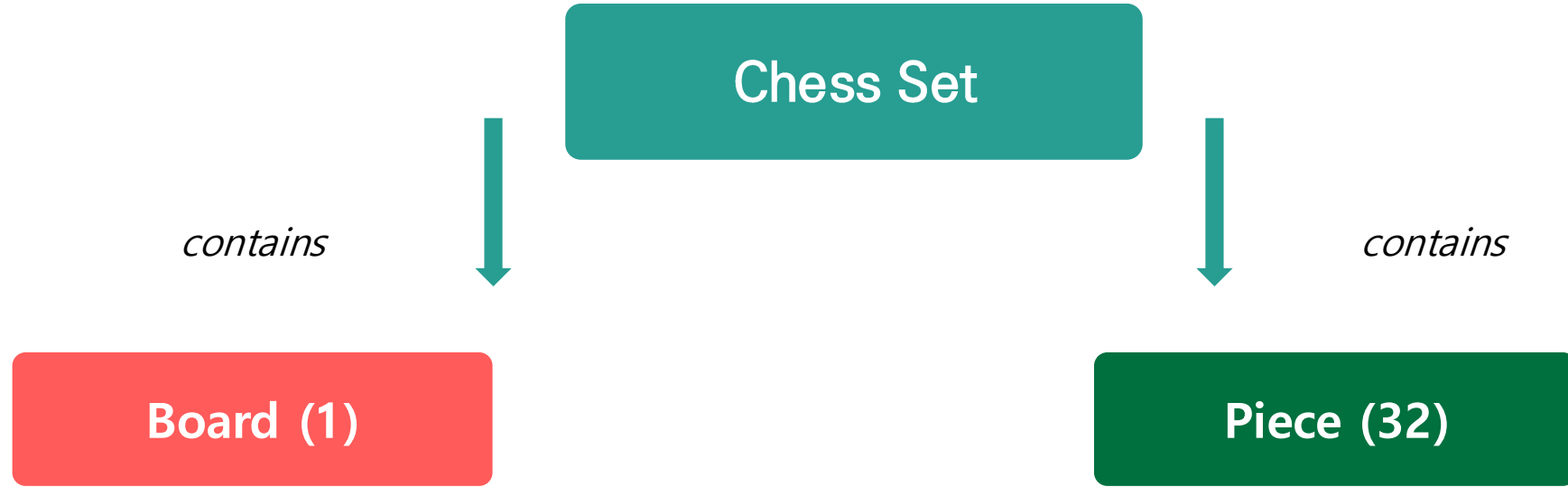
The Problem

The calling object assumes responsibility for internal logic, bypassing safety checks.

The Solution

Use the public interface: `tv.volume_up()`

Composition



Composition is the act of collecting several objects together to create a new, complex object. It represents a **"has-a"** relationship.

Composition vs. Aggregation

Composition (Solid Diamond)

Definition: Strict "part-of". Lifespans are tied. Destroying parent destroys children.

Example: Chess Set contains Board (Board cannot exist without the set).

Aggregation (Hollow Diamond)

Definition: Loose "collection". Items exist independently.

Example: Chess Set aggregates Players (Players exist before and after the game).

The "Is-A" Relationship

Inheritance

- Allows a new class (subclass) to inherit attributes and behaviors from an existing class (superclass).

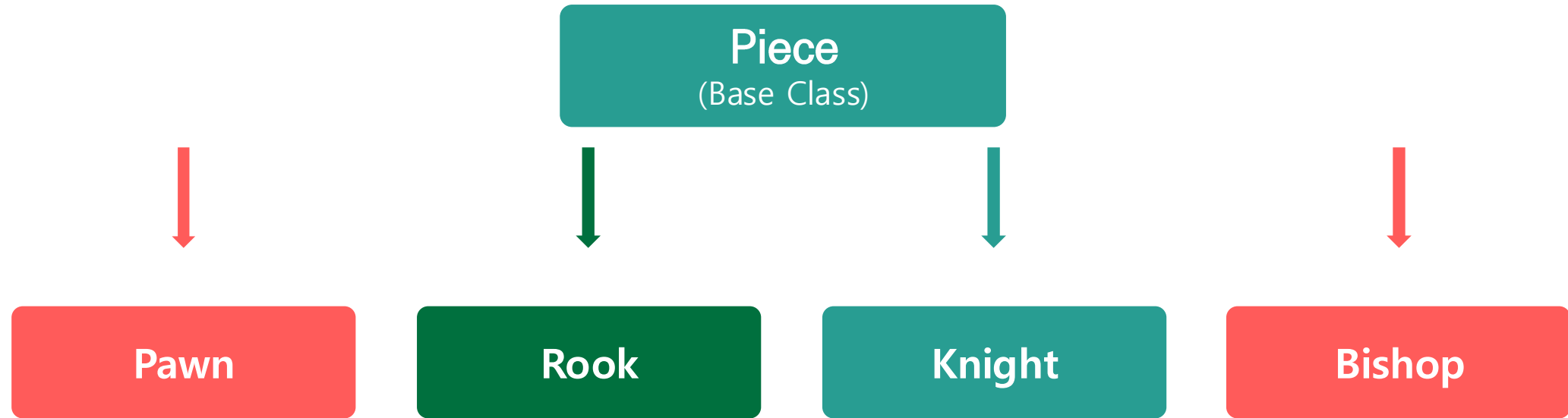
Analogy: Family Tree

- Just as you inherit your grandfather's traits, a subclass inherits from its parent.

Key Benefit

- Prevents code duplication: common logic is written once in the parent and reused by all children.
- Defines an **Is-A** relationship.

A Chess Class Hierarchy



Deep Blue **is a** Player. A Knight **is a** Piece.

All pieces share **color**, **shape**, and **move()** from the base Piece class.

Polymorphism

Definition

Polymorphism ("many forms") is the ability to treat an object differently depending on which subclass is implemented, using a shared interface.

Example

The Board calls `piece.move()`. It doesn't check if the piece is a Knight or Pawn. The specific subclass executes its unique logic automatically.

The Power of Duck Typing

Before (Procedural)

```
if type(piece) == Knight:
    move_L_shape()
elif type(piece) == Pawn:
    move_forward()
```



After (Polymorphic)

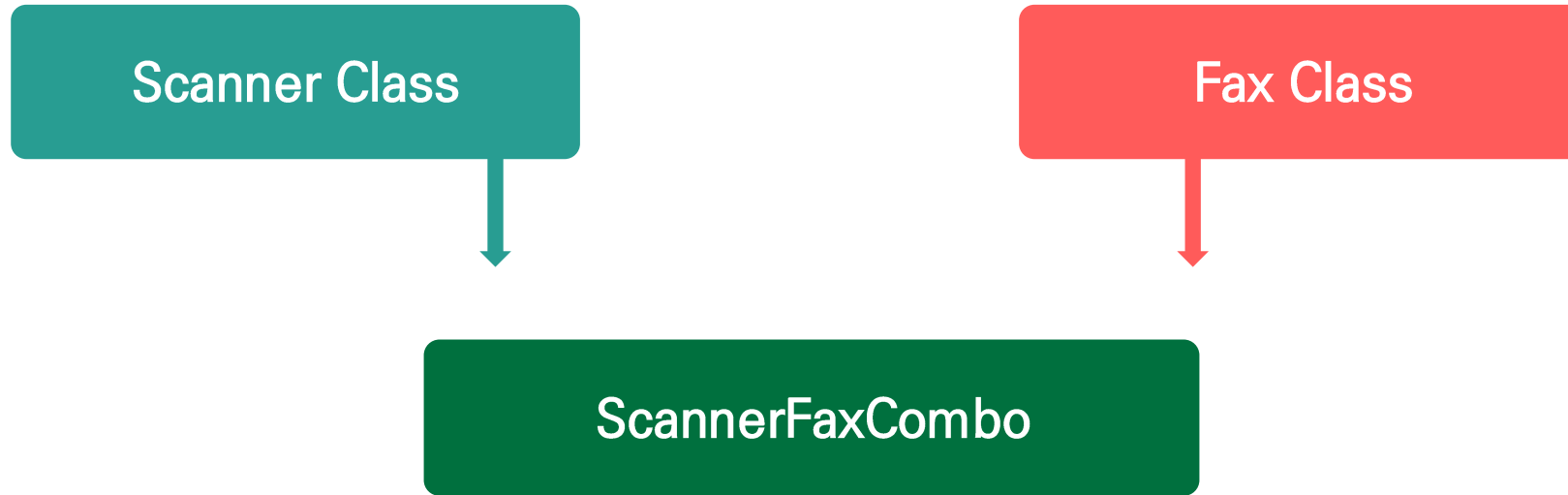
piece.move()

The object itself knows how to move. No type checking needed.

In Python, "**Duck Typing**" means if it walks like a duck and quacks like a duck, it **is** a duck. Any object with a **move()** method can participate.



Multiple Inheritance



A subclass inheriting functionality from **multiple parent classes** simultaneously.

Python allows this, unlike most traditional OO languages.

The Diamond Problem

The Danger

Uncontrolled Multiple Inheritance leads to ambiguous method resolution.

The Problem

If Class D inherits from B and C, both inheriting from A, which parent's `__init__` is called first?

The Solution

Use **Mixins** or favor **Composition** over complex inheritance trees.

Designing a System

Key Takeaway

Real-world systems require multi-faceted analysis.

We use the **"4+1" Architectural View Model** to examine a problem from multiple distinct perspectives simultaneously, ensuring no requirements are missed.

Because a change in one view generally ripples across the others, evaluating all five ensures a **robust, fully-fleshed-out** software architecture.

The 4+1 Views

1. Logical View

- Data entities, attributes, and object relationships (The heart of OOD).

2. Process View

- How data is processed (State models, Activity diagrams).

3. Development View

- Code components, modules, and packages.

4. Physical View

- Deployment and integration of the compiled application.

+1. Context View

- The unifying overview of external actors interacting with the system.

The Iris Classification Problem

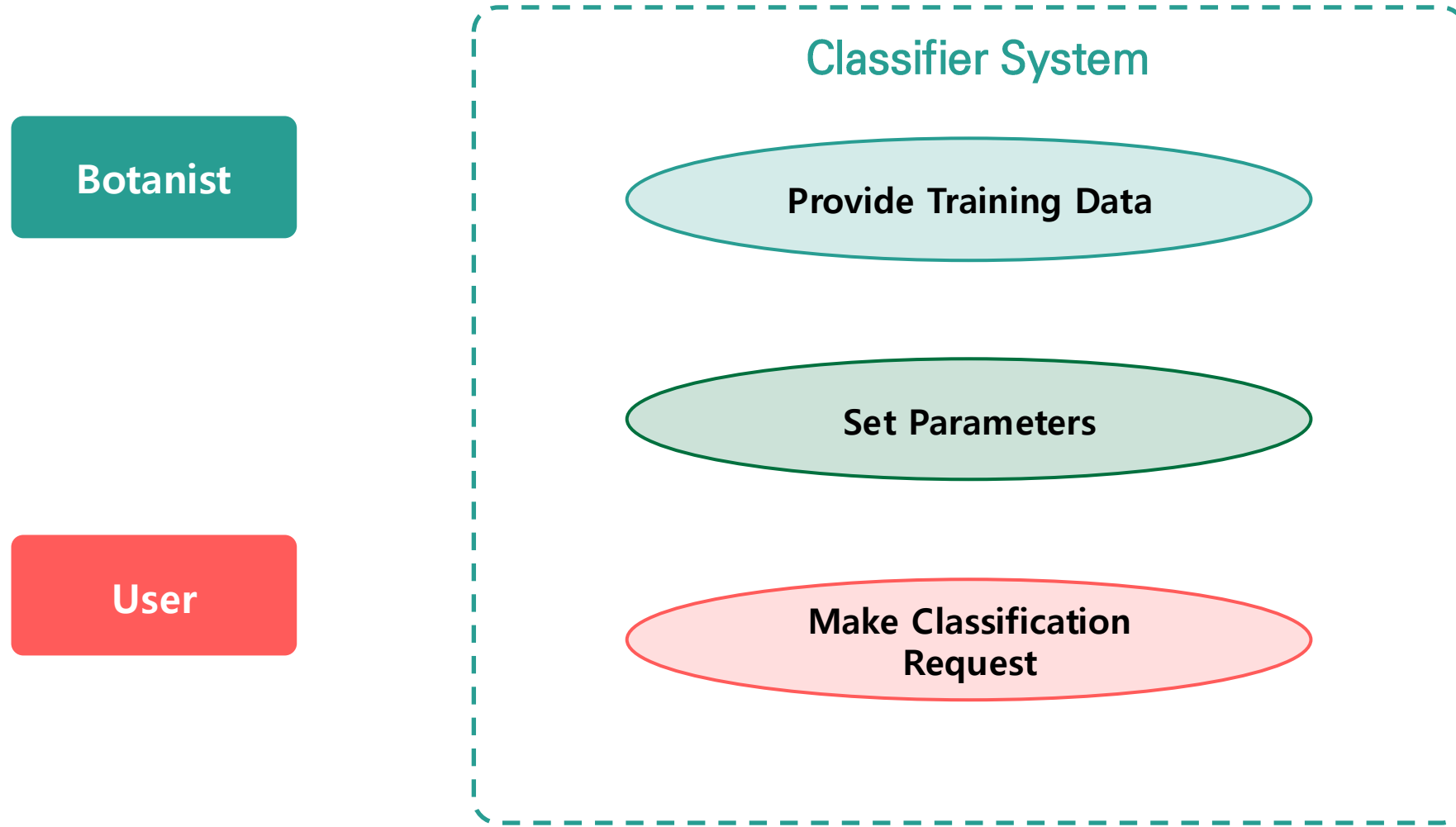
Problem

We must build a Machine Learning classifier to identify the species of Iris flowers based on physical measurements.

Algorithm

We will use the **k-Nearest Neighbors (k-NN)** algorithm, which classifies unknown data points based on their proximity to known "training" data points.

The Context View



UML Use Case Diagram

Defining User Stories

1

Provide Training Data

The Botanist uploads a dataset of known samples to establish the baseline.

2

Set Parameters

The Botanist adjusts the k value to optimize the accuracy of the model.

3

Make Classification Request

The User provides a petal/sepal measurement, and the system returns the predicted species.

The Logical View



Sample Attributes:

- sepal_length, sepal_width, petal_length, petal_width

Relationships:

- TrainingData **strictly contains** two lists (Composition).
- Sample objects **aggregate** inside these lists.

Refining the Relationships

Composition (TrainingData → Lists)

- The lists are fundamentally part of the dataset object.
- **Correct.** The lists have no meaning outside the dataset context.

Aggregation (List → Samples)

- The samples exist independently of the list.
- **Questionable.** Are samples ever used outside a list? If not, this may be over-complication.

The Process View

Activity Diagram: Upload Training Data

Start → **Validate Data**

If **Valid** → Partition Samples → Save TrainingData → **End**

If **Invalid** → Respond with error → **End**

Aligning Process and Logic

Key Takeaway

The discovery of **verbs** in the Process View dictates **methods** in the Logical View.

Our process view introduced the verb "**Partition**". This forces us to return to our Class Diagram and ensure a **partition()** method exists on the appropriate class to fulfill this responsibility.

The Development View

UML Package Diagram

Client App → imports → **REST API**

REST API → imports → **Classifier Engine**

Translating logical models into physically isolated code modules.

Modularity

Steps to structure the code:

1. Extract core data classes (Sample, TrainingData) into a **domain.py** module.
2. Isolate the k-NN algorithm into a **classifier.py** module.
3. Build web routing logic in an **api.py** module.
4. Establish clear dependency lines: api.py imports from classifier.py, but **NEVER** the reverse.

The Physical View

UML Deployment Diagram

Client Device (Browser)

-- HTTPS -->

Server Node (GUnicorn → Classifier)

How the software components map to **physical or virtual hardware**.

Scaling the Application

Key Takeaway

The Physical View exposes **constraints** that influence the Logical View.

If the system must support 100,000 requests per second, the Physical View reveals we need **multiple server nodes**. This forces a return to the Logical View to ensure our objects are **stateless** and **thread-safe**.

Conclusion of OOD

Key Takeaway

Good design is an iterative process of discovery.

Do not expect to draw a perfect UML diagram and then simply type the code. Expect to **analyze, draw, write a little code, discover a flaw**, and redraw the diagram.

Chapter 1 Recall

Q1. What is the fundamental difference between an object and a class?

Q2. What are the two primary elements that make up an object?

Q3. What is the primary difference between Composition and Aggregation?

Q4. What is the "Diamond Problem"?

Summary

Key Takeaway

OOD transforms complex problems into interacting models.

By mastering **abstraction**, **encapsulation**, **composition**, and **inheritance**, we can architect scalable, maintainable Python systems built from cleanly modeled classes.